

Eugene Moshchyn

Dr. Shangqing Zhao

CS 5173, Section 1

April 14, 2026

Homework 4

Part 1 — Questions

Problem 1 Compute the following with DETAILED MATHEMATICAL STEPS. (Hints: Use Fermat Theorem, Euler Theorem, properties of totient functions, etc)

- (1) $1234^{16} \bmod 17$
- (2) $54^{51} \bmod 17$
- (3) $\phi(51)$
- (4) $\gcd(33, 121)$
- (5) $2^{-1} \bmod 17$ (i.e., multiplicative inverse of 2 mod 17)
- (6) $\text{ind}_{2,5}(4)$
- (7) $\phi(8000)$
- (8) $\phi(98,803,519)$
- (9) $\phi(999,866,001,989)$ for the graduate session only.

Solution

- (1) $1234^{16} \bmod 17$

Solution: By Fermat's Little Theorem, since 17 is prime, we know that:

$$\begin{aligned} a^{p-1} &\equiv 1 \pmod{p} \\ 1234^{16} &\equiv 1 \pmod{17} \\ 1234^{17-1} &\equiv 1 \pmod{17} \end{aligned}$$

Therefore, the answer is 1. ■

(2) $54^{51} \bmod 17$

Solution: Since 17 is prime, by Fermat's Little Theorem, $\phi(17) = 16$. Applying modular exponentiation and then the modular multiplication rule:

$$\begin{aligned} 54^{51} \bmod 17 &= 54^{51 \bmod 16} \bmod 17 \\ &= 54^3 \bmod 17 \\ &= (54 \bmod 17)^3 \bmod 17 \\ &= 3^3 \bmod 17 \\ &= 27 \bmod 17 \\ &= 10 \end{aligned}$$

Therefore, the answer 10. ■

(3) $\phi(51)$

Solution: By Euler's Totient Function, we know that:

$$\begin{aligned} \phi(51) &= \phi(17 \times 3) \\ &= \phi(17) \times \phi(3) \\ &= 16 \times 2 \\ &= 32 \end{aligned}$$

Therefore, the answer is 32. ■

(4) $\gcd(33, 121)$

Solution: By finding the prime factorization of both numbers, we have:

$$\begin{aligned} 33 &= 3 \cdot 11 \\ 121 &= 11^2 \end{aligned}$$

The greatest prime factor shared by both numbers is 11. Therefore, the answer to $\gcd(33, 121)$ is 11. ■

(5) $2^{-1} \bmod 17$ (i.e., multiplicative inverse of 2 mod 17)

Solution: Since 17 is prime and 2 is not divisible by 17, by Fermat's Little Theorem, $2^{17-1} \equiv 1 \pmod{17}$. This implies that $2 \cdot 2^{15} \equiv 1 \pmod{17}$, making the

multiplicative inverse $2^{15} \bmod 17$. Applying modular exponentiation rules:

$$\begin{aligned}
 2^{-1} \bmod 17 &= 2^{15} \bmod 17 \\
 &= (2^4)^3 \cdot 2^3 \bmod 17 \\
 &= 16^3 \cdot 8 \bmod 17 \\
 &= (-1)^3 \cdot 8 \bmod 17 \\
 &= -8 \bmod 17 \\
 &= 9
 \end{aligned}$$

Therefore, the answer is 9. ■

(6) $\text{ind}_{2,5}(4)$

Solution: By using properties of discrete logarithms, we can get the following:

$$\begin{aligned}
 \text{ind}_{2,5}(4) &= \text{ind}_{2,5}(2^2) \\
 &= (2 \times \text{ind}_{2,5}(2)) \bmod \phi(5) \\
 &= (2 \times \text{ind}_{2,5}(2)) \bmod 4 \\
 &= (2 \times 1) \bmod 4 \\
 &= 2 \bmod 4 \\
 &= 2
 \end{aligned}$$

Therefore, the answer is 2. ■

(7) $\phi(8000)$

Solution: By Euler's Totient Function, we know that:

$$\begin{aligned}
 \phi(8000) &= \phi(8 \times 1000) \\
 &= \phi(2^3 \times 10^3) \\
 &= \phi(2^3 \times (5 \cdot 2)^3) \\
 &= \phi(2^3 \times 5^3 \times 2^3) \\
 &= \phi(2^{3+3} \times 5^3) \\
 &= \phi(2^6 \times 5^3) \\
 &= \phi(2^6) \times \phi(5^3) \\
 &= ((2-1) \cdot 2^5) \times ((5-1) \cdot 5^2) \\
 &= (1 \cdot 32) \times (4 \cdot 25) \\
 &= 32 \times 100 \\
 &= 3200
 \end{aligned}$$

Therefore, the answer is 3200. ■

(8) $\phi(98,803,519)$

Solution: First, let's find the prime factorization of 98,803,519:

$$98,803,519 = 9,949 \times 9,931$$

Both 9,949 and 9,931, hence we can use Euler's Totient Function properties for distinct primes:

$$\begin{aligned}\phi(98,803,519) &= \phi(9,949 \times 9,931) \\ &= \phi(9,949) \times \phi(9,931) \\ &= 9,948 \times 9,930 \\ &= 98,783,640\end{aligned}$$

Therefore, the answer is 98,783,640. ■

(9) $\phi(999,866,001,989)$ **for the graduate session only.**

Solution: First, let's find the prime factorization of 999,866,001,989:

$$999,866,001,989 = 999,883 \times 999,983$$

Both 999,883 and 999,983, hence we can use Euler's Totient Function properties for distinct primes:

$$\begin{aligned}\phi(98,803,519) &= \phi(999,883 \times 999,983) \\ &= \phi(999,883) \times \phi(999,983) \\ &= 999,882 \times 999,982 \\ &= 999,864,002,124\end{aligned}$$

Therefore, the answer is 999,864,002,124. ■

Note

To determine whether an integer is prime and to extract its factors, the following Python functions were used:

```

1 def is_prime(number: int) -> bool:
2     if number < 2:
3         return False
4
5     for i in range(2, int(number ** 0.5) + 1):
6         if number % i == 0:
7             return False
8
9     return True

```

Listing 1: The `is_prime` function: Determines if a given integer is prime.

```

1 import math
2
3
4 def get_factors(number: int) -> list[int]:
5     factors = set()
6
7     for i in range(1, int(math.sqrt(number)) + 1):
8         if number % i == 0:
9             factors.add(i)
10            factors.add(number // i)
11
12     return sorted(list(factors))

```

Listing 2: The `print_factors` function: Calculates and displays the factors of an integer.

Problem 2 Describe the differences between digital signature and digital certificate.

Solution

A digital signature is a cryptographic mechanism used to verify the authenticity and integrity of a message, document, or piece of data.

A digital certificate is a digital document (structured file) issued by a trusted third party called a Certificate Authority (CA). Its purpose is to bind a public key to an identity (a person, organization, or device).

Feature	Digital Signature	Digital Certificate
Nature	Cryptographic operation or output	Structured data file or document
Purpose	Proves authenticity and integrity of data	Binds a public key to a specific identity
Created by	The message sender (using their private key)	A trusted Certificate Authority (CA)
Protects	A specific message or document	A public key and its owner's identity
Trust anchor	Mathematical proof (asymmetric cryptography)	A trusted third party (the CA)

Problem 3 Describe the differences between PKI and KDC.

Solution

PKI (Public Key Infrastructure) – is a decentralized system using asymmetric cryptography. A Certificate Authority (CA) signs certificates that bind a public key to an identity, which anyone can verify without contacting the CA in real time. This makes it highly scalable and well-suited for open environments like the internet (e.g., HTTPS/TLS).

KDC (Key Distribution Center) – is a centralized system using symmetric cryptography (e.g., Kerberos). A central server shares a secret key with every user/service and issues session keys when two parties want to communicate. The KDC must be online during authentication, making it a single point of failure, but it works very efficiently in closed networks like corporate Active Directory environments.

Feature	PKI	KDC
Cryptography	Asymmetric	Symmetric
Architecture	Decentralized	Centralized
Scalability	High(internet-scale)	Lower(domain-bound)
Best suited for	Open, large-scale networks	Closed, trusted networks

Problem 4 Describe the differences between symmetric crypto and asymmetric crypto.

Solution

Symmetric cryptography uses a single shared key for both encryption and decryption. Because it is computationally faster, it is ideal for encrypting bulk data. However, it requires both parties to securely agree on and exchange the key beforehand.

In contrast, asymmetric cryptography uses a mathematically linked key pair: a public key that can be openly distributed for encryption, and a private key kept secret for decryption. While slower, it solves the key distribution problem. Therefore, asymmetric cryptography is typically used for secure key exchanges, digital signatures, and encrypting smaller amounts of sensitive data.

Problem 5 Explain the reflection attacks against symmetric key based authentication, and explain potential countermeasures.

Solution

Reflection Attack – is an attack against symmetric key based authentication protocols where an attacker exploits a challenge-response exchange by opening a parallel session and reflecting the server's own challenge back at it, tricking it into computing the correct response. Since both parties share the same key and the protocol does not distinguish message direction or session context, the attacker uses the server as an unwitting oracle to authenticate without ever knowing the shared secret.

To prevent reflection attacks, the overarching principle is to ensure the two parties do not do exactly the same thing. Specific countermeasures include:

- **Initiator proves identity first:** The protocol should require the initiator of the communication to authenticate themselves before the responder does, operating on the assumption that the initiator is more likely to be the attacker.
- **Directional key separation (Different Keys):** Use distinct keys for each communication direction (e.g., $K_{Alice-Bob} = K_{Bob-Alice} + 1$), so a response encrypted for one direction cannot be reused in the other.
- **Asymmetric challenges (Different Challenges):** Structure challenges so Alice and Bob's challenges cannot be the same, meaning they use provably different formats.
- **Session binding:** Tag messages explicitly with session IDs and roles to establish context, preventing cross-session reuse of messages.

Problem 6 Suppose we are using a three-message mutual authentication protocol, and Alice initiates contact with Bob. Suppose we wish Bob to be a stateless server, and therefore it is inconvenient to require him to remember the challenge he sent to Alice. Let's modify the exchange so that Alice sends the challenge back to Bob, along with the encrypted challenge. So the protocol is shown as follows. Is this protocol secure? Why?

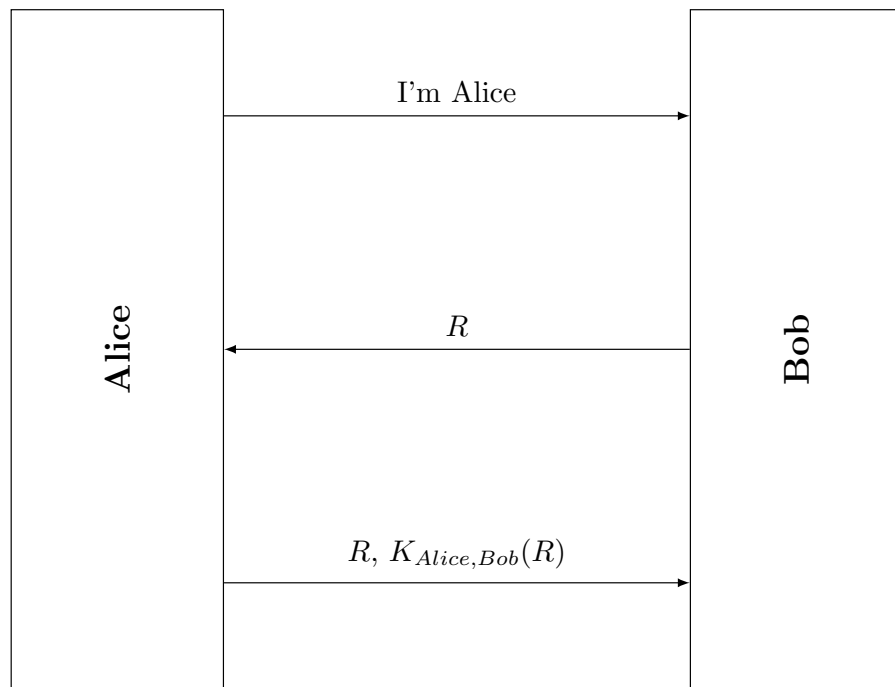


Figure 1: Mutual Authentication Protocol Sequence Diagram

Solution

The protocol is **NOT** secure. It is vulnerable to a **replay attack**, which compromises the authentication between the two parties. Because Bob is a stateless server and does not remember the challenge he originally sent, an attacker can replay a previous message to trick Bob into believing they are Alice.

Problem 7

Let's modify the protocol from the previous problem so that Bob (still stateless) sends both a challenge, and a challenge encrypted with a key K_{Bob} that only he knows. Is this protocol secure? Why?

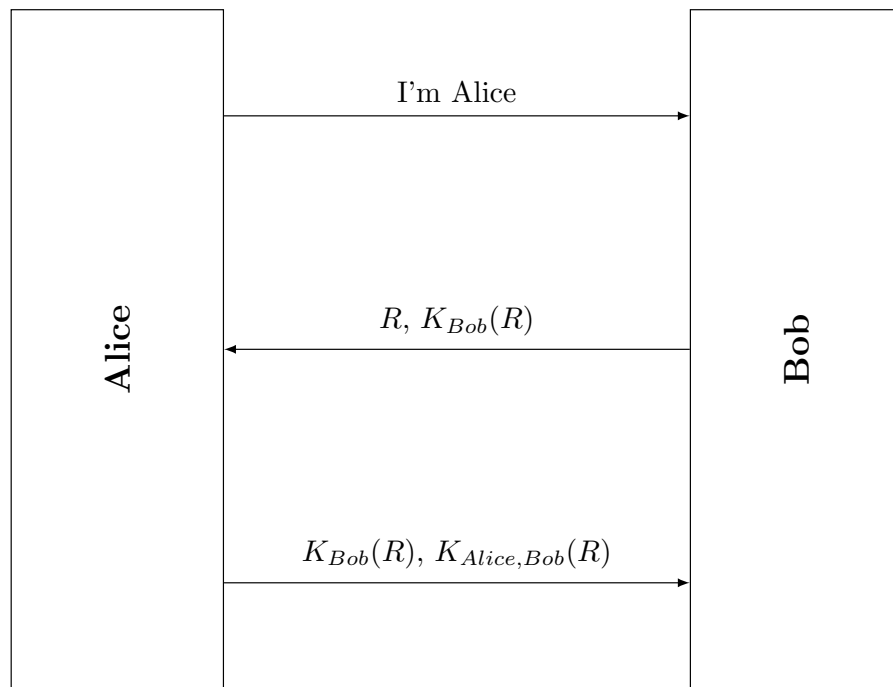


Figure 2: Modified Mutual Authentication Protocol Sequence Diagram

Solution

The protocol is **NOT** secure. It is also vulnerable to a **replay attack**, which compromises the authentication between the two parties. Although Bob attempts to handle his statelessness by sending an encrypted state token alongside the challenge, he still cannot verify the freshness of the response. An attacker can replay a previously captured message containing the old token and encrypted challenge to trick Bob into believing they are Alice.

Problem 8

Suppose Alice and Bob know each other's public key, and they use the following protocol to achieve mutual authentication: Alice signs the message $M_1 = \text{"This is Alice"}$ and Bob signs the message $M_2 = \text{"This is Bob"}$. Explain the vulnerability.

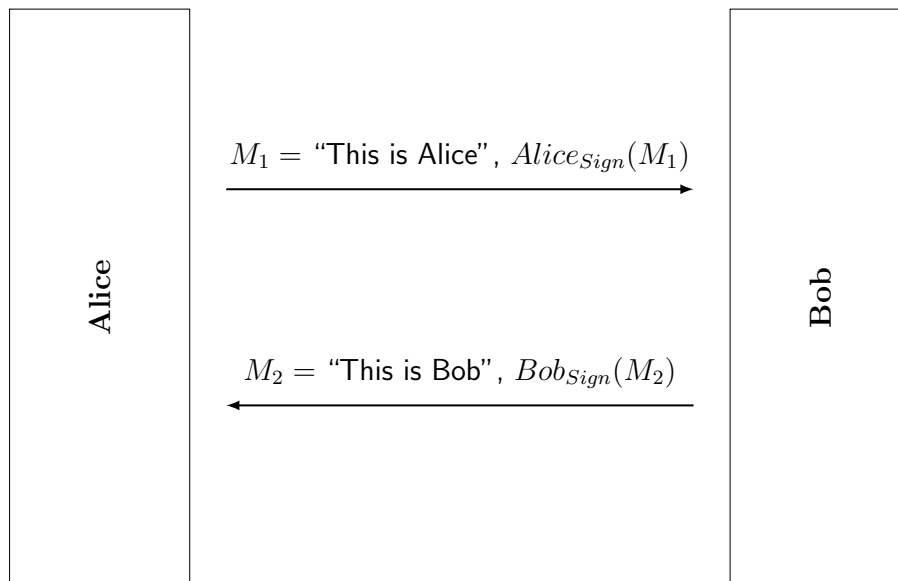


Figure 3: Mutual Authentication Exchange Between Alice and Bob

Solution

The protocol is **NOT** secure. In this protocol, Alice signs the static message $M_1 = \text{"This is Alice"}$ and Bob signs the static message $M_2 = \text{"This is Bob"}$. The critical vulnerability here is the complete lack of a mechanism to guarantee the “freshness” of the session.

- An attacker (Trudy) can passively monitor the network and capture Alice’s transmission containing M_1 and her digital signature.
- At any later point in time, Trudy can initiate a new connection with Bob and send that exact same copied message.
- Bob will use Alice’s public key to verify the signature. Because the signature is mathematically valid for the string “This is Alice”, Bob will successfully verify it and falsely authenticate Trudy as Alice.
- This design fails because there is no challenge involved to prove Alice is actively generating a signature for this specific session at this specific time.

Problem 9

Now, suppose Alice and Bob use random numbers to enhance security. Alice signs the combination of message $M_1 = \text{"This is Alice"}$ and a random number R_1 , then send M_1 , R_1 , and digital signature to Bob; and Bob does the similar thing. Is this secure?

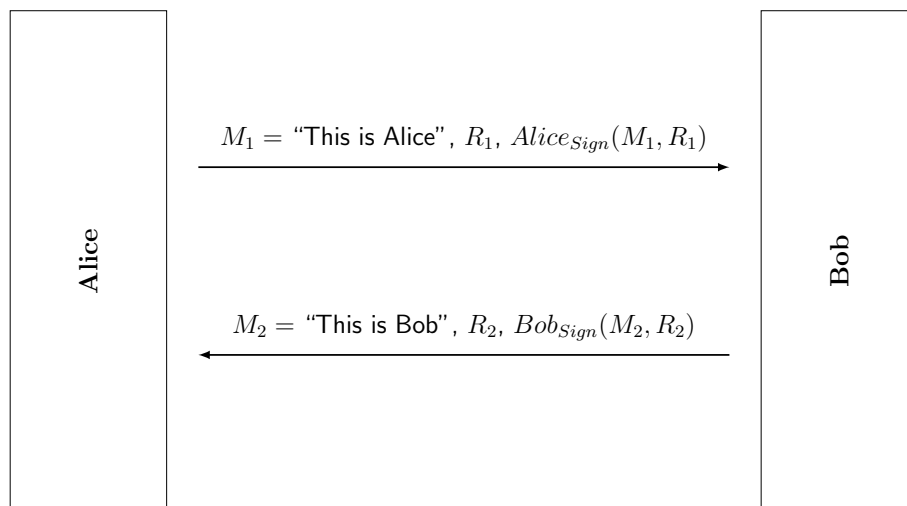


Figure 4: Mutual Authentication Exchange Between Alice and Bob

Solution

The protocol is **NOT** secure. The protocol is modified so Alice signs the combination of $M_1 = \text{"This is Alice"}$ and a random number R_1 . Bob does the same with M_2 and a random number R_2 . While introducing random numbers is a step in the right direction, this specific implementation is still vulnerable to a replay attack.

- For a random number (nonce) to effectively prevent replay attacks in a challenge-response system, the verifier must generate the challenge.
- In this design, Alice generates her own random number R_1 , packages it with M_1 , and signs the whole thing before sending it to Bob.
- Because Bob did not issue R_1 as a challenge to Alice, he has no way of knowing if this is a fresh random number generated right now, or an old one from a previous legitimate session.
- Trudy can simply intercept the entire package (M_1 , R_1 , and the signature) from a past session and send it to Bob later.
- Unless Bob is designed to perfectly memorize every single R_1 Alice has ever sent him to check for duplicates, which contradicts the principles of stateless server design mentioned elsewhere in your assignment, Bob's system will verify the signature against the provided M_1 and R_1 , and falsely accept the replayed message.

Problem 10

An even trickier question: In a three-message authentication protocol, Alice initiates contact with Bob. Bob is a stateless server, and thus it is inconvenient for him to remember the challenges sent to Alice. Assume that Alice and Bob share a key $K_{\text{Alice-Bob}}$ and Bob also has a key K_{Bob} only known to himself. The protocol works as follows, where R_1 and R_2 are random numbers generated by Alice and Bob, respectively.

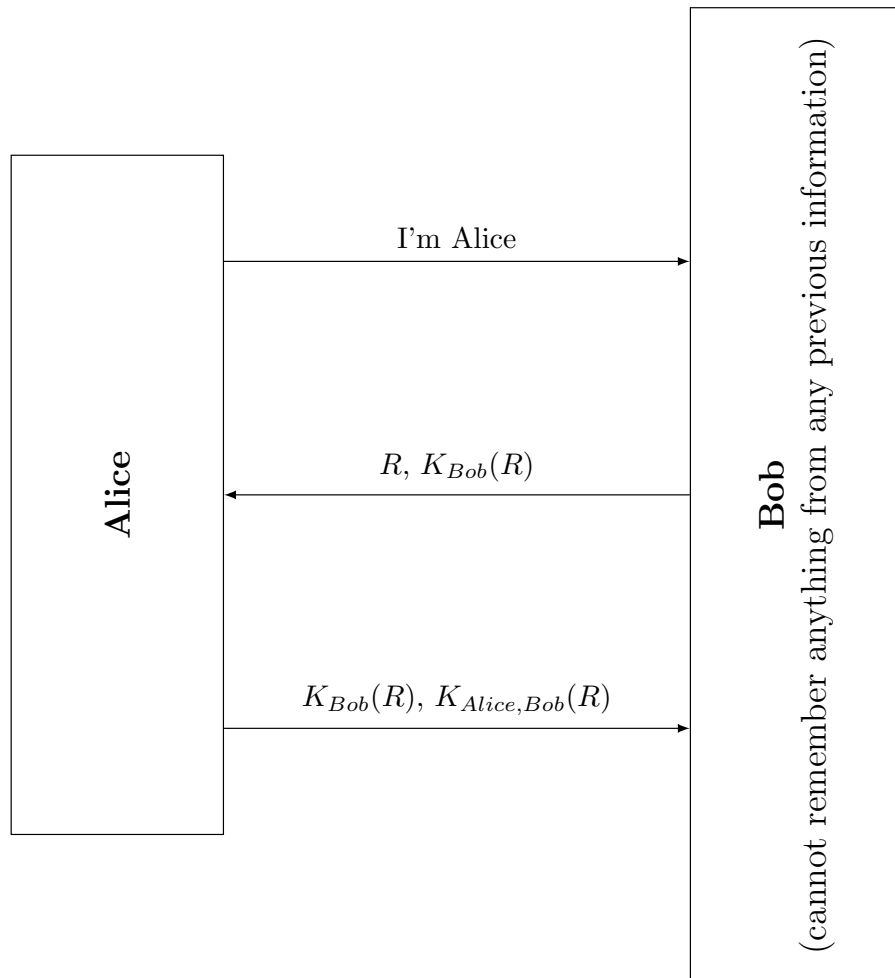


Figure 5: Mutual Authentication Exchange Between Alice and Bob

- Does this protocol provide mutual authentication? If yes, why? If no, who cannot authenticate whom and why?
- Could a third person, Trudy, impersonate Alice? Describe a possible attack scenario. If it is not possible, clearly state so and give reasons.
- Could Trudy impersonate Bob? Describe a possible attack scenario. If it is not possible, clearly state so and give reasons.

Solution

Does this protocol provide mutual authentication? If yes, why? If no, who cannot authenticate whom and why

Yes, under normal, honest execution. If we assume no active attackers are interfering, the protocol is theoretically designed to achieve mutual authentication.

- (a) Alice authenticates Bob: In the first message, Alice sends a fresh random challenge, R_1 . In the second message, Bob replies with $K_{Alice-Bob}(R_1 + R_2)$. Because this ciphertext contains Alice's fresh R_1 (modified by adding R_2) and is encrypted with their shared secret key, Alice has proof that the sender possesses the key and is actively responding to her specific session.
- (b) Bob authenticates Alice: Bob sends his own challenge, R_2 , hidden inside the encrypted state $K_{Bob}(R_2)$ and the shared encryption $K_{Alice-Bob}(R_1 + R_2)$. To successfully generate the final message, $K_{Alice-Bob}(R_2)$, Alice must possess the shared key to decrypt the payload, extract R_2 , and re-encrypt it. When Bob receives this, he verifies it using his stateless ticket, proving Alice holds the key

Could a third person, Trudy, impersonate Alice? Describe a possible attack scenario. If it is not possible, clearly state so and give reasons.

Yes. Trudy can easily impersonate Alice by exploiting how Bob generates his response, effectively turning Bob into an encryption oracle. This is a classic chosen-plaintext attack.

Attack scenario:

- (a) The Exploit: The protocol dictates that Bob takes Alice's R_1 , adds his R_2 , and encrypts the result.
- (b) The Attack: Trudy initiates contact with Bob, claiming to be Alice. However, instead of sending a random number for R_1 , she deliberately sends $R_1 = 0$.
- (c) Bob's Oracle Response: Bob receives 0, generates his random R_2 , and computes $0 + R_2$, which is simply R_2 . He then replies to Trudy with $K_{Bob}(R_2)$ and $K_{Alice-Bob}(R_2)$.
- (d) The Impersonation: Trudy now holds exactly what she needs for the third message. She simply echoes Bob's message back to him: $K_{Bob}(R_2)$ and $K_{Alice-Bob}(R_2)$.
- (e) Result: Bob decrypts his state ticket, checks the payload, and successfully authenticates Trudy as Alice, even though Trudy has no knowledge of the shared key $K_{Alice-Bob}$.

Could Trudy impersonate Bob? Describe a possible attack scenario. If it is not possible, clearly state so and give reasons.

Yes. Trudy can impersonate Bob to Alice by using an interleaving/relay attack (a variation of the reflection attacks discussed in your lecture notes). Because the encrypted payloads do not explicitly bind the messages to a specific sender or direction, Trudy can use the real Bob to solve Alice's challenges for her.

Attack scenario:

- (a) Interception: Alice initiates a legitimate session, sending I'm Alice, R_1 to Trudy (believing Trudy is Bob).
- (b) Relay (Session 2): Trudy pauses Alice, turns around, and initiates a new parallel session with the real Bob, sending him Alice's exact message: I'm Alice, R_1 .

- (c) Bob's Honest Response: The real Bob processes this normally, generating his own R_2 , and replies to Trudy with $K_{Bob}(R_2)$ and $K_{Alice-Bob}(R_1 + R_2)$.
- (d) The Impersonation: Trudy forwards this exact message back to Alice in their original session.
- (e) Result: Alice decrypts the payload, verifies her R_1 is present, and falsely concludes she is communicating securely with Bob. Trudy has successfully impersonated Bob without ever knowing the keys.

Problem 11 What is the TCP/IP protocol? Specify what is the purpose of this protocol, and what kind of information it provides.

Solution

The Internet Protocol (IP) is the address system of the Internet and has the core function of delivering packets of information from a source device to a target device. IP is the primary way in which network connections are made, and it establishes the basis of the Internet. IP does not handle packet ordering or error checking. Such functionality requires another protocol, often the Transmission Control Protocol (TCP) [2].

Together, the TCP/IP protocol provides critical routing and delivery information, including source and destination IP addresses, port numbers to identify specific applications, and sequence numbers to reassemble data correctly.

Problem 12 What is DNS, and how can ISP know the domain information that you want to visit?

Solution

The Domain Name System (DNS) is the phonebook of the Internet. Humans access information online through domain names, like `nytimes.com` or `espn.com`. Web browsers interact through Internet Protocol (IP) addresses. DNS translates domain names to IP addresses so browsers can load Internet resources.

Each device connected to the Internet has a unique IP address, which other machines use to find the device. DNS servers eliminate the need for humans to memorize IP addresses such as 192.168.1.1 (in IPv4), or more complex newer alphanumeric IP addresses such as 2400:cb00:2048:1::c629:d7a2 (in IPv6) [1].

- Unencrypted DNS: By default, your router uses your ISP's DNS servers. Because standard DNS requests are unencrypted, the ISP simply reads the requested domain names in plain text.
- SNI Data (Server Name Indication): Even on secure (HTTPS) websites, your browser must announce the specific domain name it wants to connect to in plain text during the initial security handshake.

- **Destination IP Routing:** Because the ISP physically routes all data, they must read the destination IP address on every packet to deliver it, which reveals the server you are communicating with.

Problem 13 What is the difference between using proxy server and the Tor?

Solution

A proxy server functions as an intermediary between a client device and the internet, forwarding requests such that the destination server observes the proxy's IP address rather than the originating device's. This architecture, however, introduces a single point of trust: the proxy operator retains visibility into the client's original IP address, browsing destinations, and any unencrypted traffic. While proxies offer low latency and are effective for circumventing geo-restrictions or network-level access controls, they provide limited anonymity and presuppose confidence in the operator's integrity.

Tor is designed with a fundamentally different threat model, prioritizing trustless anonymity over performance. Rather than routing traffic through a single intermediary, Tor applies multiple layers of encryption and distributes the connection across at least three volunteer-operated relays — an entry, middle, and exit node. Each relay decrypts only the layer necessary to identify the next hop, ensuring that no individual node possesses both the client's identity and the requested destination. This multi-hop architecture substantially mitigates the risks of traffic analysis and network surveillance, at the cost of increased latency inherent to the additional routing complexity.

Problem 14 Why proof of work is necessary to build a blockchain? Suppose if we remove the proof of work and design a simple rule: the first node in the network that finishes computing a new block always wins and this new block will be immediately broadcast to the network and all nodes must add this block to the chain. How will this simple (yet flawed) design impact the security?

Solution

Proof of Work (PoW) serves as a protocol to verify computational effort and acts as a safeguard to prevent malicious actors from compromising the ledger. Because generating specific hashes is resource-intensive, it becomes both computationally and financially prohibitive to add fraudulent blocks to the blockchain. This process requires nodes to solve complex mathematical puzzles, ensuring that no single entity can easily dominate the network.

Without the PoW, several security issues arise:

- **Sybil Attacks:** Without PoW, a malicious actor could create a large number of fake nodes at little to no cost, allowing them to gain disproportionate influence over the network's consensus mechanism.

- **Double-Spending:** Without the computational cost of PoW acting as a deterrent, an attacker could potentially broadcast two conflicting transactions simultaneously, spending the same funds more than once before the network can reach consensus.
- **Race Conditions:** Multiple valid blocks could be broadcast to the network at nearly the same time, creating ambiguity about which transaction should be considered final, and allowing bad actors to exploit the window between submission and confirmation.
- **Forks:** Without PoW enforcing a single canonical chain, the network would have no reliable mechanism to resolve competing chain branches, leading to persistent splits in the ledger and inconsistent transaction histories across nodes.
- **Security:** The overall integrity of the blockchain would be severely weakened – PoW ensures that rewriting history (e.g., a 51% attack) requires an enormous amount of computational resources, making such attacks economically impractical. Without it, the cost of attacking the network drops dramatically.

Part 2 — Extra Credit

Problem 15 The expanded Needham-Schroeder Protocol can be shortened to a 6-message protocol without loss of security by merely removing the 7th message. Why is this true? (Hint: the purpose of the 7th message is to prove to Bob that he is talking to Alice, but he already knows that. Why?)
 (10 extra credit points for both graduates and undergraduates)

Solution

In the Expanded Needham-Schroeder Protocol, the seventh message, intended for Bob to prove his active communication with Alice, is entirely redundant. Bob has already authenticated Alice in the sixth message by using his secret key to decrypt her ticket and reveal her name. Because this successfully confirms her identity to Bob, the seventh message can be safely removed from the protocol without compromising its security.

References

- [1] Cloudflare. What is dns? — how dns works, 2026.
- [2] Cloudflare. What is TCP/IP?, 2026.